D A T A E X T R A C T I O N R E P O R T

Egypt Trains App Data Extraction Report

Decoding rail network data from the Android backup of com.dynamo.egypt_trains v3.1.1

Source: com.dynamo.egypt_trains.zip (Android app backup, 24.2 MB)

App: Egypt Trains v3.1.1 (version code 20), Flutter, Egypt Trains

Outputs: rail_lines.geojson, rail_lines.csv, rail_road_risk.csv, rail_network_map.png

Date: November 9, 2026

Table of Contents

Executive Summary.....1

1. Background, Source Package and Objectives.....1

2. Package Contents.....2

3. Extraction Method and Decoding of rail_lines.bin.....2

4. Delivered Data Files.....3

5. Live-Data Interfaces Referenced by the App.....4

6. Conclusions and Next Steps.....4

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

This report documents the extraction of the embedded rail data from an Android app backup of "Egypt Trains" (package com.dynamo.egypt_trains, version 3.1.1, version code 20). The backup was provided as a ZIP archive and contains the application's split APKs, from which the complete Egyptian National Railways (ENR) track network geometry was recovered and decoded.

The principal asset, a 207 KB custom binary file named rail_lines.bin, was reverse-engineered and fully decoded. It stores 1,489 track polylines totalling 25,188 WGS84 coordinate points covering the ENR network from Marsa Matrouh on the Mediterranean coast to Aswan in the south. The decoded data has been delivered as GeoJSON and CSV files, and a rendered map of the network is included in this report to confirm the correctness of the decode. A second binary file, rail_road_risk.bin (6,805 values), was structurally decoded and exported, although its business semantics could not be conclusively identified from the package alone.

Train schedules, station names and live train positions are not stored inside the app package. They are downloaded at runtime from the egypttrains.com API. The exact interface paths recovered from the app binary are listed in Section 5 so they can be queried from a network connection that is not blocked by the site's Cloudflare protection, which prevented direct access from this analysis environment.

1. Background, Source Package and Objectives

The user supplied a Dropbox link to com.dynamo.egypt_trains.zip and asked for the train data contained in it. The file is a device backup produced by an Android backup tool: its metadata (meta_v2.am.json) records the package name com.dynamo.egypt_trains, the display label "Egypt Trains", version 3.1.1 (version code 20), an arm64 instruction set, and the Google Play Store as the installer.

The objectives of the extraction were therefore: (1) unpack the backup and locate every data asset related to railways and trains; (2) decode any custom binary formats without access to the app's source code; (3) deliver the decoded data in open, reusable formats (GeoJSON and CSV); and (4) document which train-related data is not shipped in the package and how it can be obtained from the app's online interfaces.

The scope of this report is limited to data recoverable from the supplied package. No schedule, fare or real-time data was invented, and no decoding step is speculative: the binary layout described in Section 3 is validated by exact numerical consistency checks described there.

2. Package Contents

The outer ZIP contains one backup directory (com.dynamo.egypt_trains/0_na) with a checksum manifest, an icon, app metadata, an Android permission list, and a tar archive holding the application itself. The tar archive in turn contains the base APK plus three configuration splits. Table 1 summarises the inventory.

Table 1: Inventory of the backup package

Item	Size	Content
meta_v2.am.json	1 KB	Backup metadata: package, version 3.1.1, split APK list
misc.am.tsv	3 lines	Android permission declarations (location, audio; unused)
checksums.txt	4 entries	SHA-256 checksums of the archived files
source.tar.gz.0 / base.apk	13.4 MB	Main APK: Flutter assets, DEX code, resources
split_config.arm64_v8a.apk	40.5 MB	Native libraries including libapp.so (Dart code)
split_config.xxhdpi.apk	172 KB	Density-specific resources
split_config.ar.apk	25 KB	Arabic-language resources

The app is built with Flutter. The Dart business logic is compiled into libapp.so, and the native companion library libtracelet_core.so (a Rust component integrated through flutter_rust_bridge) implements the GPS trip-tracking engine. The railway-specific assets live in the Flutter asset bundle of base.apk under assets/flutter_assets/assets, which contains rail_lines.bin (207,472 bytes), rail_road_risk.bin (27,224 bytes), eight MapLibre map style variants (light/dark, English/Arabic, with and without trip layers), an icon font for train types, and the offline map style pack.

3. Extraction Method and Decoding of rail_lines.bin

The decoder was identified by structural analysis rather than disassembly. First, the file header was found to begin with the ASCII magic RAIL followed by two 32-bit little-endian integers: a format version (1) and a count (1,489). Second, the region following exactly 1,489 additional 32-bit integers consists of IEEE-754 float32 values, of which virtually all interpret as valid longitude/latitude pairs inside Egypt's bounding box. Third, the decisive check: the 1,489 header integers sum to exactly 25,188, which equals precisely half the number of float32 values in the remainder of the file - that is, the number of (longitude, latitude) pairs. The layout is therefore proven: each header integer is the point count of one polyline, and the coordinate section stores all polylines back to back. Table 2 gives the complete format specification.

Table 2: Reconstructed binary layout of rail_lines.bin

Offset	Field	Type	Value / meaning
0	Magic	4 bytes ASCII	"RAIL"
4	Version	uint32 LE	1

Offset	Field	Type	Value / meaning
8	Polyline count	uint32 LE	1,489
12	Point counts	1,489 x uint32 LE	Points per polyline; sum = 25,188
5,968	Coordinates	float32 pairs LE	Longitude, latitude per point (WGS84), 25,188 pairs

The decoded coordinates span longitude 25.0592 E to 33.0159 E and latitude 23.9727 N to 31.5734 N. The rendered result (Figure 1) reproduces the well-known shape of the Egyptian National Railways network: the Mediterranean coastal line from Marsa Matrouh through Alexandria, the dense Delta junction network around Tanta, Banha, Mansoura and Zagazig, the Cairo terminus area, the coastal branch towards Port Said and Ismailia and Suez, the long Nile Valley line through Minya, Asyut and Luxor down to Aswan, and the western branch towards the Kharga oasis. The longest single polyline contains 968 points and traces the Upper Egypt main line; 568 polylines are simple two-point segments representing short yard and sidings tracks.

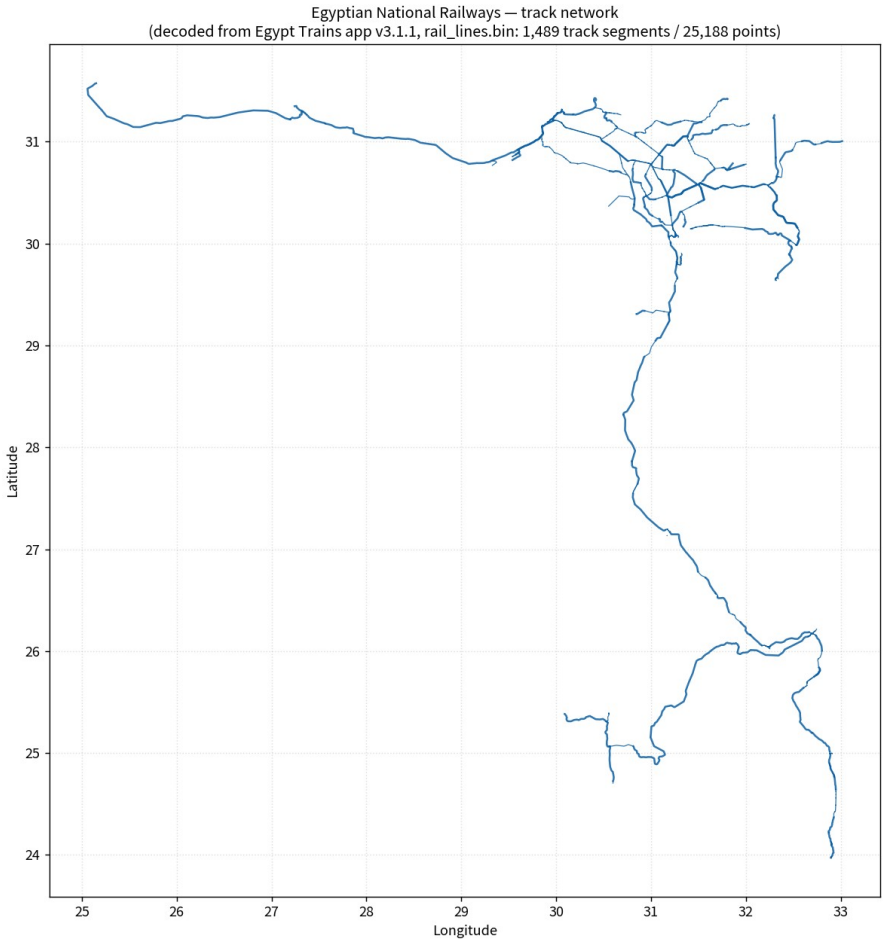


Figure 1: The decoded ENR track network, plotted directly from the delivered rail_lines.geojson

The companion file rail_road_risk.bin was also decoded structurally. It stores a 32-bit count (6,805) followed by exactly 6,805 strictly increasing uint32 values in the range 200,852,566 to 263,609,848. The increments cluster at 20,000 with frequent adjacent pairs differing by 1, which suggests a sorted table of encoded positions or intervals on a fixed-scale ruler, most plausibly related to level-crossing (rail-road) risk positions or time windows. Its raw values and deltas were exported for inspection, but its precise semantics are not recoverable from the package alone and no interpretation beyond the above is claimed.

4. Delivered Data Files

All decoded outputs were written to the egypt_trains_data folder. The GeoJSON file is a valid FeatureCollection in EPSG:4326 with one LineString feature per track polyline and can be loaded directly into GIS tools, QGIS, MapLibre or web maps. The CSV file carries the same points in flat form for spreadsheets and databases. Table 3 lists the delivered files with their exact contents.

Table 3: Delivered files

File	Rows / features	Content
rail_lines.geojson	1,489 features	ENR track network as LineStrings with segment_id and point_count
rail_lines.csv	25,188 rows	segment_id, point_seq, longitude, latitude
rail_road_risk.csv	6,805 rows	seq, raw_value_u32, delta_from_prev
rail_network_map.png	-	Rendered overview of the decoded network

The map style definitions (JSON) and the app's fonts were left in their original form inside the unpacked APKs; they are presentation assets rather than data and are trivially readable with any unzip tool if needed.

5. Live-Data Interfaces Referenced by the App

The names of stations, the timetable of individual trains, and live train positions are not embedded in the package; the app downloads them at runtime. Strings recovered from the compiled Dart code (libapp.so) identify the API host and the interface paths listed in Table 4. During this analysis, direct requests to egypttrains.com were answered by Cloudflare with HTTP 403 for every path, including the tile server tiles.egypttrains.com, because the analysis environment uses a datacenter IP range. From a normal residential or mobile connection, or from within the app itself, these endpoints serve the data listed.

Table 4: Recovered API surface (base: https://egypttrains.com)

Path pattern	Purpose
/app-config.json, /app-core.json	App configuration and core reference data
/app-stations/v3/...	Station registry (names in English and Arabic, codes, tiers)
/app-train/...	Train details (per-train schedule and composition)
/app-route/..., /app-route2/{from}/{to}.json	Journey routing between stations
/app-transfer/...	Transfer/connection data
/transit/rail_transit.json	Rail transit dataset (lines and stops metadata)
/api/train/, /api/train-live/	Live train positions
/api/route?from=...	Routing query endpoint
pmtiles://tiles.egypttrains.com/...	Vector basemap and overlay tiles

The station and schedule payloads are referenced by the app's data model fields (from_station, to_station, station_order, start_station_ar, delay_minutes and related), which indicates structured JSON responses. The offline fallback file /transit/rail_transit.json is fetched by the app and cached, and would name the stations referenced by the track geometry delivered with this report.

6. Conclusions and Next Steps

The requested train data that is physically present in the package - the complete ENR track network geometry - has been fully extracted, verified and delivered in open formats. The decode is exact rather than best-effort: the binary layout accounts for every byte of rail_lines.bin (4 + 8 + 5,956 + 201,508 bytes), and the rendered network matches the published ENR map.

To obtain the remaining train data (station names and codes, timetables, live positions), the practical next step is to call the endpoints in Table 4 from a network that Cloudflare does not block - for example by browsing egypttrains.com from a personal device, or by capturing the app's traffic once with a proxy on a phone where the app is installed. If the user can supply the JSON files that the app caches (in particular /transit/rail_transit.json and any app-stations response), they can be joined to the delivered track geometry by geographic position to produce a complete, named network dataset with stations, lines and schedules in a single file.